



Software Factory en la Era de la IA

Herramientas

Organización	lean-ai
Clasificación	Generado desde sf-kb
Fecha	13 de julio de 2026
Versión	1.0

Ides Editores

Visión General

El IDE (Integrated Development Environment) es la herramienta primaria del developer. La elección correcta impacta directamente productividad, code quality y experiencia del equipo.

Panorama Actual

Categorías de Herramientas

Categoría	Ejemplos	Características
IDE completo	JetBrains IntelliJ, Visual Studio	Compilación, debugging profundo, refactoring
Editor extensible	VS Code, Sublime Text	Lightweight, ecosistema de plugins, AI integrado
AI-native IDE	Cursor, Windsurf	IA como ciudadano de primera clase
Terminal-based	Vim/Neovim, Helix, Zed	Speed, minimal footprint, workflows teclado

Comparativa de Principales Herramientas

VS Code

Aspecto	Detalle
Base	Electron (Chromium + Node.js)
Costo	Gratuito, open-source
Lenguajes	Multi-lenguaje via extensions
AI Integration	GitHub Copilot, Continue, Codeium
Remote Dev	Dev Containers, SSH, WSL
Marketplace	+40,000 extensiones
Rendimiento	Medio en proyectos grandes

Fortalezas: Ecosistema masivo, integración Git nativa, debugging avanzado, terminal integrada.

Debilidades: Consumo de memoria, rendimiento con proyectos monolíticos.

JetBrains IntelliJ /全家桶

Aspecto	Detalle
Base	JVM (IntelliJ Platform)
Costo	Community (gratis) / Ultimate (paid)
Lenguajes	IDEs específicos por lenguaje
AI Integration	JetBrains AI Assistant
Refactoring	Best-in-class
Debugging	Enterprise-grade
Rendimiento	Alto consumo, pero optimizado

Fortalezas: Refactoring inteligente, análisis de código profundo, database tools, Spring support.

Debilidades: Costo licenciamiento, consumo de recursos.

Cursor

Aspecto	Detalle
Base	Fork de VS Code
Costo	Free tier + Pro (\$20/mes)
AI Features	Chat, Composer, Tab completions, codebase indexing
Modelos	GPT-4, Claude, custom
Codebase Awareness	Indexa todo el proyecto
Privacy	Mode without code retention

Fortalezas: AI contextualizada en el proyecto completo, Composer para cambios multi-archivo.

Debilidades: Newer, smaller ecosystem than VS Code, dependencia de conectividad AI.

Windsurf (Codeium)

Aspecto	Detalle
Base	Fork de VS Code
Costo	Free tier generoso
AI Features	Cascade (multi-step agentic), autocomplete
Differentiator	Flows — AI que navega entre archivos

Fortalezas: Agentic workflow sin fricción, free tier competitivo.

Vim / Neovim

Aspecto	Detalle
Base	Terminal
Costo	Gratuito, open-source
Modal Editing	Normal, Insert, Visual, Command
Plugins	lazy.nvim, packer
AI Integration	Codeium.nvim, Copilot.vim, Avante
Learning Curve	Alto al inicio, productividad máxima después
Rendimiento	Excepcional

Fortalezas: Velocidad, no depende de GUI, funciona en cualquier entorno, personalización total.

Debilidades: Curva de aprendizaje empinada, configuración inicial compleja.

Feature Matrix

Feature	VS Code	JetBrains	Cursor	Windsurf	Neovim
IntelliSense					⚙️
AI Autocomplete					⚙️
AI Chat					⚙️
AI Agentic	⚙️	⚙️			⚙️
Refactoring					⚙️
Debugging					⚙️
Git Integration					⚙️
Remote Dev					
Performance	⚙️	⚙️	⚙️	⚙️	
Extensiones					

(*best-in-class*, *good*, ⚙️ *via plugins/config*)

Configuración Recomendada por Stack

Backend Java/Kotlin

- **IDE:** IntelliJ IDEA Ultimate
- **Plugins:** Key Promoter, String Manipulation, SonarLint

Frontend/Full-Stack

- **IDE:** VS Code o Cursor
- **Extensions:** ESLint, Prettier, Tailwind CSS, GitLens

Data Engineering

- **IDE:** VS Code + Jupyter, PyCharm Professional
- **Extensions:** Python, Jupyter, SQLTools

DevOps/Infrastructure

- **IDE:** VS Code o Neovim
- **Extensions:** Terraform, Docker, Kubernetes, YAML

Decision Framework

¿Necesitas AI nativo? → Cursor o Windsurf
¿Equipo enterprise con budget? → JetBrains
¿Ecosistema y comunidad? → VS Code
¿Maximum performance? → Neovim/Zed
¿Budget limitado? → VS Code + Codeium

Impacto en Prácticas de Equipo

- **Estándarización:** Usar mismo IDE facilita pairing y code review
- **Settings sync:** VS Code y JetBrains ofrecen sync de configuración
- **Dev Containers:** Estandarizar entorno con Docker reduces "works on my machine"
- **Linting integrado:** Configurar herramientas de calidad en el IDE
- Referencia: Procesos de desarrollo

Recursos

- VS Code Tips: <https://code.visualstudio.com/docs>
- JetBrains Academy: <https://www.jetbrains.com/academy/>
- Neovim: <https://neovim.io/doc/>
- Cursor Docs: <https://docs.cursor.sh>

Herramientas Ci Cd

Visión General

CI/CD es el backbone del desarrollo moderno. Estas herramientas automatizan build, test y deployment, habilitando prácticas como trunk-based development y delivery continuo.

Arquitectura Típica de un Pipeline

Code Push → Build → Unit Tests → Integration Tests → Security Scan → Deploy Staging → E2E Tests

Componentes Clave

Componente	Propósito	Ejemplos
CI Server	Ejecuta pipelines	Jenkins, GitHub Actions
Artifact Registry	Almacena builds	Docker Hub, Nexus, JFrog
Secrets Management	Gestiona credenciales	Vault, AWS Secrets Manager
Notification	Alerta resultados	Slack, Teams, Email
Infrastructure	Agents/Runners	Self-hosted, cloud

Comparativa de Herramientas CI/CD

GitHub Actions

Aspecto	Detalle
Hosted	GitHub cloud runners
Self-hosted	runners en cualquier infra
Pricing	Free tier generoso (2000 min/mes)
YAML	Workflow syntax
Marketplace	+20,000 actions
Re-usable	Composite actions, reusable workflows
Matrix builds	nativo
Secrets	Encrypted, org-level

```
# Ejemplo workflow básico
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
      - run: npm ci
      - run: npm test
```

Fortalezas: Deep integration con GitHub, ecosistema masivo, fácil de empezar.

Debilidades: Vendor lock-in a GitHub, debugging workflows puede ser difícil.

Jenkins

Aspecto	Detalle
Modelo	Self-hosted, open-source
Plugins	+1,800 plugins
Configuration	Jenkinsfile (Groovy DSL)
Distributed	Master-agent architecture
Pipeline as Code	declarative y scripted
Extensibilidad	Máxima
Learning Curve	Alto

Fortalezas: Control total, extensibilidad ilimitada, madurez enterprise.

Debilidades: Mantenimiento pesado, UI obsoleta, curva de aprendizaje alta.

GitLab CI/CD

Aspecto	Detalle
Integration	Nativo en GitLab
Runners	Shared + specific
Auto DevOps	Pipeline automático
Container Registry	Integrado
Security Scanning	SAST, DAST, dependency scanning incluido

Aspecto	Detalle
Environments	Review apps, staging, production
Pricing	Included in GitLab tiers

Fortalezas: All-in-one platform, security scanning built-in, auto DevOps.

Debilidades: Completo pero pesado, self-hosted requiere resources.

CircleCI

Aspecto	Detalle
Speed	Caching agresivo, parallelism
Orbs	Paquetes reutilizables
Docker	First-class support
SSH Debug	interactuar con jobs
Pricing	Generoso free tier

Fortalezas: Velocidad, debugging SSH, caching inteligente.

Debilidades: Menos ecosistema que GitHub Actions.

ArgoCD (GitOps)

Aspecto	Detalle
Modelo	GitOps declarativo
Kubernetes	Native K8s delivery
Sync	Auto or manual
UI	Visual diff de estado deseado vs actual
Multi-cluster	

Fortalezas: GitOps puro, visual, multi-cluster.

Debilidades: Solo Kubernetes, requires GitOps maturity.

Comparison Matrix

Feature	GitHub Actions	Jenkins	GitLab CI	CircleCI	ArgoCD
Cloud-hosted		✗			✗
Self-hosted					

Feature	GitHub Actions	Jenkins	GitLab CI	CircleCI	ArgoCD
Free tier					
Marketplace					⚙️
Security scanning	⚙️	⚙️		⚙️	✖️
GitOps	⚙️	⚙️		✖️	
Learning curve	Baja	Alta	Media	Baja	Media
Maintenance	Baja	Alta	Media	Baja	Media
K8s native	⚙️	⚙️			

Decision Framework

¿Ya usas GitHub? → GitHub Actions
 ¿Necesitas control total? → Jenkins
 ¿Quieres all-in-one? → GitLab CI
 ¿Velocidad es prioridad? → CircleCI
 ¿GitOps + Kubernetes? → ArgoCD
 ¿Enterprise strict? → Jenkins o GitLab CI self-hosted

Mejores Prácticas

Pipeline Design

- **Fast feedback:** Unit tests primero, E2E al final
- **Parallelización:** Dividir jobs independientes
- **Caching:** Dependencies, build artifacts
- **Fail fast:** Stops en primer error significativo

Security en CI/CD

- **Secrets scanning** en repositorio
- **SAST** en cada pull request
- **Dependency scanning** automático
- **Container scanning** para imágenes Docker
- Referencia: Testing tools

Pipeline como Código

- Versionado con el código fuente
- Revisado en code review
- Testeado (self-testing pipelines)
- Documentado con README del repo

Métricas de CI/CD

Métrica	Target	Referencia	Métricas]]
Build duration	< 10 min	[[../06-Metricas/README	
Deployment frequency	Diaria+	DORA metrics	
Lead time for changes	< 1 día	DORA metrics	
Change failure rate	< 5%	DORA metrics	
Mean time to recovery	< 1 hora	DORA metrics	

Integración con Otras Herramientas

- **Testing**: Herramientas testing — Ejecutar tests en pipeline
- **IaC**: Iac terraform — Deploy infra desde CI/CD
- **Containers**: Container kubernetes — Build y push imágenes
- **Monitoring**: Herramientas monitoring — Post-deploy verification
- **AI**: Ai coding assistants — AI en pipeline (review, test gen)

Gestion Repositorios

Visión General

Los repositorios son el sistema central de verdad para código fuente. La plataforma elegida define flujos de code review, CI/CD integration, y colaboración del equipo.

Plataformas Principales

GitHub

Aspecto	Detalle
Users	100M+ developers
Repos	420M+ repositorios
Pricing	Free, Team (\$4/user/mes), Enterprise (\$21/user/mes)
CI/CD	GitHub Actions (ver Herramientas ci cd)
Package Registry	GitHub Packages (npm, Docker, Maven)
Security	Dependabot, Code Scanning, Secret scanning
AI	GitHub Copilot integration

Features Clave:

- Branch protection rules
- CODEOWNERS
- Required reviewers
- Status checks
- Auto-merge
- Pull request templates
- Issue templates
- GitHub Projects (project management)
- GitHub Advanced Security

GitLab

Aspecto	Detalle
Modelo	Self-hosted + SaaS
Pricing	Free, Premium (\$29/user/mes), Ultimate (\$99/user/mes)

Aspecto	Detalle
CI/CD	GitLab CI/CD nativo
Security	SAST, DAST, dependency scanning, container scanning
Wiki	Built-in
Package Registry	Built-in (multiple formats)
Milestone	Integrated DevSecOps

Features Clave:

- All-in-one DevOps platform
- Auto DevOps
- Review Apps
- Environment management
- Feature flags
- Value stream analytics

Bitbucket

Aspecto	Detalle
Owner	Atlassian
Pricing	Free (5 users), Standard (\$3/user/mes), Premium (\$6/user/mes)
CI/CD	Bitbucket Pipelines
Integration	Deep Jira, Trello integration
Branch Permissions	Granular
Code Insights	

Features Clave:

- Jira integration (commit linking)
- Smart mirroring
- IP whitelisting (Premium)
- Merge checks

Comparison Matrix

Feature	GitHub	GitLab	Bitbucket
Free private repos			
Built-in CI/CD			

Feature	GitHub	GitLab	Bitbucket
Security scanning			
Container registry			
Wiki			
Project management			 (Jira)
Self-hosted	(GHES)		
AI features	(Copilot)		
Marketplace			
Learning curve	Baja	Media	Baja

Branch Protection Rules

Configuración Recomendada

```
# Configuración mínima para production repos
branch_protection:
  main:
    required_reviews: 2
    dismiss_stale: true
    require_code_owner_reviews: true
    required_status_checks:
      - ci/build
      - ci/test
      - security/scan
    enforce_admins: true
    restrict_pushes: true
    allow_force_pushes: false
    allow_deletions: false
```

Estrategias de Branching

Estrategia	Uso	Branches
Trunk-based	Equipos ágiles, CI/CD maduro	main + short-lived feature branches
GitFlow	Release cycles largos	main, develop, feature/, release/, hotfix/*
GitHub Flow	Deploy continuo	main + feature branches
GitLab Flow	Environment-based	main + environment branches

Referencia: Estrategias de branching

CODEOWNERS

```
# Ejemplo CODEOWNERS
*.js           @frontend-team
*.java         @backend-team
*.tf           @platform-team
/docs/         @tech-writing
/Dockerfile    @devops-team
*.test.js      @qa-team
```

Beneficios:

- Auto-assign reviewers por path
- Required reviews de owners específicos
- Escalación automática de conocimiento
- Integra con quality gates

Features de Security

Dependabot / Renovate

- Auto-update dependencias
- Security alerts
- Version management

Secret Scanning

- Detecta secrets committed accidentalmente
- Pre-commit hooks como primera línea
- GitHub push protection

Code Scanning (SAST)

- CodeQL en GitHub
- Semgrep integration
- Referencia: AI Code Review

Gestión de Monorepos

Herramientas

Herramienta	Tipo	Use Case
Nx	Build system	JavaScript/TypeScript monorepos
Turborepo	Build system	JS/TS, caching inteligente

Herramienta	Tipo	Use Case
Bazel	Build system	Polyglot, escalable
Lerna	Package management	npm monorepos
Rush	Package management	Enterprise npm

Decisiones Monorepo vs Multirepo

Factor	Monorepo	Multirepo
Deploy atómico		✗
Code sharing		⚙️
CI complexity	Alta	Baja
Team isolation	Baja	
Tooling cost	Alto	Bajo

Métricas de Repositorio

Métrica	Target	Herramienta
PR review time	< 4 horas	GitHub Insights
PR size	< 400 LOC	Guidelines
Code churn	< 20%	Analytics
Bus factor	≥ 2	CODEOWNERS analysis
Test coverage	> 80%	Herramientas testing

Integración CI/CD

- Configurar pipelines en cada push
- Status checks obligatorios antes de merge
- Auto-deploy a staging tras merge a main
- Referencia: Procesos de deployment

Recursos

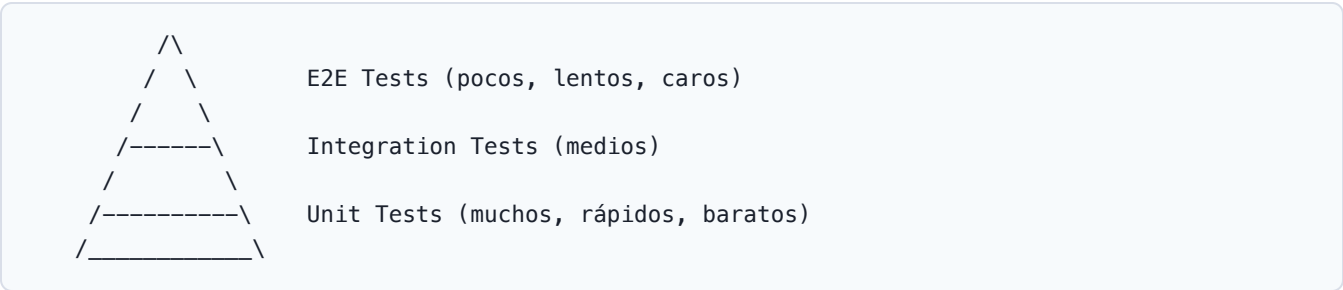
- GitHub Docs: <https://docs.github.com>
- GitLab Docs: <https://docs.gitlab.com>
- Atlassian Git Tutorials: <https://www.atlassian.com/git>

Herramientas Testing

Visión General

Las herramientas de testing automatizado son pilares de la calidad de software. Cada tipo de test tiene herramientas optimizadas, y la elección correcta depende del stack tecnológico, layer de testing y contexto del equipo.

Testing Pyramid



Unit Testing Frameworks

JavaScript/TypeScript: Jest vs Vitest

Feature	Jest	Vitest
Speed	Medio	Rápido (Vite)
Compatibility	Universal	Growing
Watch mode		
ESM support	⚙️ Config	Native
UI testing	Jest DOM	Happy DOM
Snapshot		
Coverage		
API Compatible	—	Jest-compatible

Recomendación: Vitest para proyectos nuevos con Vite; Jest para existentes.

Java: JUnit 5 + Mockito

Feature	Detalle
Annotations	@Test, @BeforeEach, @AfterEach
Parameterized	@ParameterizedTest

Feature	Detalle
Nested	@Nested for test grouping
Extensions	@ExtendWith
Mocking	Mockito integration
Assertions	AssertJ preferred over built-in

Python: pytest

Feature	Detalle
Fixtures	@pytest.fixture — scoped, parametrized
Parametrize	@pytest.mark.parametrize
Plugins	800+ (xdist, cov, mock, httpx)
Assertions	Plain assert with diff output
Coverage	pytest-cov
Markers	@pytest.mark.slow, custom

Ventaja clave: Fixtures system es superior a unittest setUp/tearDown.

Integration Testing

API Testing

Herramienta	Tipo	Lenguaje
REST Assured	Library	Java
httpx / requests	Library	Python
Supertest	Library	Node.js
Postman/Newman	CLI/Collection	Any
Karate	DSL	Any (Gherkin)

Database Testing

Herramienta	Uso
Testcontainers	Spin up real DBs in Docker
H2	In-memory DB for Java

Herramienta	Uso
Faker	Data generation
DBUnit	Dataset management

```
// Testcontainers example
@Testcontainers
class UserRepositoryTest {
    @Container
    static PostgreSQLContainer<?> postgres = new PostgreSQLContainer<>("15");

    @Test
    void shouldSaveUser() {
        // Test with real PostgreSQL instance
    }
}
```

E2E / UI Testing

Comparison Matrix

Feature	Cypress	Playwright	Selenium	Puppeteer
Speed			⚙️	
Multi-browser	⚙️ (WebKit beta)			❌ (Chrome)
Multi-language	❌ (JS only)			❌ (JS)
Auto-wait			❌	⚙️
Network intercept			⚙️	
Parallel execution	(paid)	(free)	⚙️	⚙️
Component testing			❌	❌
Learning curve	Baja	Media	Alta	Media
Mobile testing	❌	(Appium)	(Appium)	❌
Visual regression	⚙️ (plugins)	(toHaveScreenshot)	⚙️	⚙️
Debugging	Time-travel	Trace Viewer	⚙️	⚙️

Cypress

Best for: Frontend apps, component testing, developer experience.

```
describe('Login', () => {
  it('should login successfully', () => {
    cy.visit('/login')
    cy.get('[data-cy=username]').type('user@test.com')
    cy.get('[data-cy=password]').type('password')
    cy.get('[data-cy=submit]').click()
    cy.url().should('include', '/dashboard')
  })
})
```

Playwright

Best for: Multi-browser testing, API testing, complex scenarios.

```
# Playwright Python example
def test_login(page):
    page.goto("/login")
    page.fill("[data-cy=username]", "user@test.com")
    page.fill("[data-cy=password]", "password")
    page.click("[data-cy=submit]")
    assert "/dashboard" in page.url
```

Selenium

Best for: Legacy systems, wide browser support, established teams.

Visual Regression Testing

Herramienta	Tipo	Integración
Chromatic	Cloud	Storybook
Percy	Cloud	Any
Playwright	Built-in	Playwright
BackstopJS	Local	Any
Ioki	Local	Storybook

Performance Testing

Herramienta	Tipo	Uso
k6	Load testing	API, web, microservices
JMeter	Load testing	Enterprise, protocol-level
Lighthouse	Performance audit	Web vitals

Herramienta	Tipo	Uso
Artillery	Load testing	Modern, config-driven
Locust	Load testing	Python-based, distributed

Test Management

Herramienta	Tipo	Precio
TestRail	Test management	Paid
Zephyr	Jira plugin	Paid
qTest	Enterprise	Paid
Xray	Jira native	Paid
Manual markdown	Low-tech	Free

Coverage Tools

Herramienta	Lenguaje	Target
Istanbul/nyc	JS/TS	> 80%
coverage.py	Python	> 85%
JaCoCo	Java	> 80%
Codecov	Any	Dashboard
Coveralls	Any	Dashboard

Referencia: Métricas de coverage

Decision Framework

Frontend JS/TS + Vite?	→ Vitest + Playwright
Frontend React legacy?	→ Jest + Cypress
Java Spring Boot?	→ JUnit 5 + Mockito + Testcontainers
Python/Django/FastAPI?	→ pytest + httpx
Multi-language enterprise?	→ Selenium Grid + k6
API-first microservices?	→ Karate + k6

Integración con CI/CD

- Ejecutar unit tests en cada commit
- Integration tests en PR
- E2E tests en staging deployment

- Performance tests weekly o pre-release
- Referencia: Pipelines CI/CD

Recursos

- Cypress Docs: <https://docs.cypress.io>
- Playwright: <https://playwright.dev>
- pytest: <https://docs.pytest.org>
- JUnit 5: <https://junit.org/junit5/>
- k6: <https://k6.io/docs/>

Herramientas Monitoring

Visión General

Monitoreo y observabilidad son esenciales para operar sistemas confiables. Siguiendo los principios de medición, un stack completo cubre métricas, logs y traces — los tres pilares de la observabilidad.

Los Tres Pilares de Observabilidad

Pilar	Qué captura	Herramientas típicas
Metrics	Agregaciones numéricas over time	Prometheus, Datadog
Logs	Eventos discretos con contexto	ELK, Loki
Traces	Request flow entre servicios	Jaeger, Zipkin, Tempo

Métricas: Prometheus + Grafana

Prometheus

Aspecto	Detalle
Modelo	Pull-based metrics collection
Query	PromQL
Storage	Local TSDB + remote write
Alerting	Alertmanager
Escalabilidad	Federation, Thanos, Cortex
Costo	Open-source, self-hosted

Conceptos clave:

- **Metrics:** counters, gauges, histograms, summaries
- **Scrape interval:** Default 15s
- **Service discovery:** Kubernetes, Consul, EC2
- **Recording rules:** Pre-compute expensive queries

```
# Ejemplo PromQL
# Error rate por servicio
rate(http_requests_total{status=~"5.."}[5m])
/ rate(http_requests_total[5m])
```

```
# P99 latency
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))
```

Grafana

Aspecto	Detalle
Función	Visualization + dashboarding
Data sources	Prometheus, Loki, Tempo, ES, SQL, etc.
Alerting	Unified alerting
Plugins	+180 data sources, panels
On-call	Grafana OnCall (free)
Costo	Open-source + Cloud tiers

Dashboards recomendados:

- **RED metrics:** Rate, Errors, Duration
- **USE metrics:** Utilization, Saturation, Errors
- **SLI/SLO:** Error budget tracking
- Referencia: Definición de SLIs

Logs: ELK Stack vs Loki

ELK Stack (Elasticsearch, Logstash, Kibana)

Aspecto	Detalle
Indexing	Full-text search indexing
Storage	High (inverted index)
Query	KQL, Lucene, ESQL
Costo	Alto (storage + compute)
Features	ML anomaly detection, SIEM

Grafana Loki

Aspecto	Detalle
Modelo	Index labels only, grep content
Storage	Bajo (no full-text index)
Query	LogQL (PromQL-like)

Aspecto	Detalle
Costo	Bajo
Integration	Native Grafana

Comparativa directa:

Factor	ELK	Loki
Full-text search		 grep-based
Storage cost	Alto	Bajo
Query speed		
Operational cost	Alto	Bajo
Integration	Standalone	Grafana native
Scalability		

Distributed Tracing: Jaeger vs Zipkin vs Tempo

Jaeger

Aspecto	Detalle
Origin	Uber, CNCF graduated
Storage	Cassandra, Elasticsearch
Client libs	OpenTelemetry compatible
Sampling	Adaptive, rate-based
UI	Trace visualization, comparison

Grafana Tempo

Aspecto	Detalle
Integration	Grafana native
Storage	Object storage (S3, GCS)
Costo	Bajo (no indexing)
Query	TraceQL
Exemplars	Link metrics → traces

Comparativa:

Factor	Jaeger	Zipkin	Tempo
CNCF status	Graduated	—	—
Storage cost	Medio	Medio	Bajo
Grafana integration	Plugin	Plugin	Native
OpenTelemetry			
Query language	—	—	TraceQL

APM Commercial

Datadog

Aspecto	Detalle
Coverage	Metrics + Logs + Traces + APM + RUM
Pricing	Per host + per feature
Strengths	Turnkey, 700+ integrations
AI	Watchdog anomaly detection
Cost	\$15-23/host/mo + extras

New Relic

Aspecto	Detalle
Pricing	Per-user + data ingest
Free tier	100GB/month
Strengths	Full observability, NRQL
AI	NRAI (Natural Language Queries)

Comparison: Open-source vs Commercial

Factor	Prometheus/Grafana	Datadog	New Relic
Cost	Bajo	✗ Alto	⚙ Medio
Setup effort	✗ Alto	Bajo	Bajo
Customization	Total	⚙	⚙
Support	Community	Enterprise	Enterprise

Factor	Prometheus/Grafana	Datadog	New Relic
Vendor lock-in	✗	✗ Alto	⚙️ Medio
Features			

OpenTelemetry

Aspecto	Detalle
Qué es	Standard de observabilidad (CNCF)
Signals	Traces, metrics, logs
Vendor neutral	Exporta a cualquier backend
Auto-instrumentation	Libraries para múltiples lenguajes
Collector	Proxy unificado

Recomendación: Adoptar OpenTelemetry como standard de instrumentación. Vendor-neutral, future-proof.

Stack Recomendado

Startup / Small Team

Prometheus + Grafana + Loki + Tempo + OpenTelemetry

Enterprise con budget

Datadog (full platform) o New Relic

Enterprise open-source

Prometheus + Grafana Enterprise + Loki + Tempo + OpenTelemetry

Alerting Best Practices

- Alert on symptoms, not causes
- Use SLO-based alerts
- Route alerts to correct on-call
- Avoid alert fatigue (noise reduction)
- Referencia: On-call practices

Integración

- **Kubernetes:** Container kubernetes — Exporters y sidecars
- **CI/CD:** Herramientas ci cd — Post-deploy monitoring
- **Testing:** Herramientas testing — Performance baselines
- **Métricas:** SLIs, SLOs, Error Budgets
- **AI:** AI-powered observability

Container Kubernetes

Visión General

Containers y orquestación son la base de la infraestructura moderna. Kubernetes se ha consolidado como el standard de facto para orquestación, mientras Docker remains el estándar de empaquetado.

Docker

Conceptos Fundamentales

Concepto	Descripción
Image	Blueprint inmutable de la aplicación
Container	Instancia ejecutable de una image
Dockerfile	Script de construcción de images
Volume	Persistencia de datos fuera del container
Network	Comunicación entre containers
Registry	Almacén de images (Docker Hub, ECR, GCR)

Dockerfile Best Practices

```
# Multi-stage build para reducir tamaño
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

FROM node:20-alpine AS runtime
WORKDIR /app
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
EXPOSE 3000
USER node
CMD ["node", "dist/main.js"]
```

Reglas clave:

- Usar multi-stage builds
- Minimizar capas (COPY dependencies antes de código)
- No correr como root

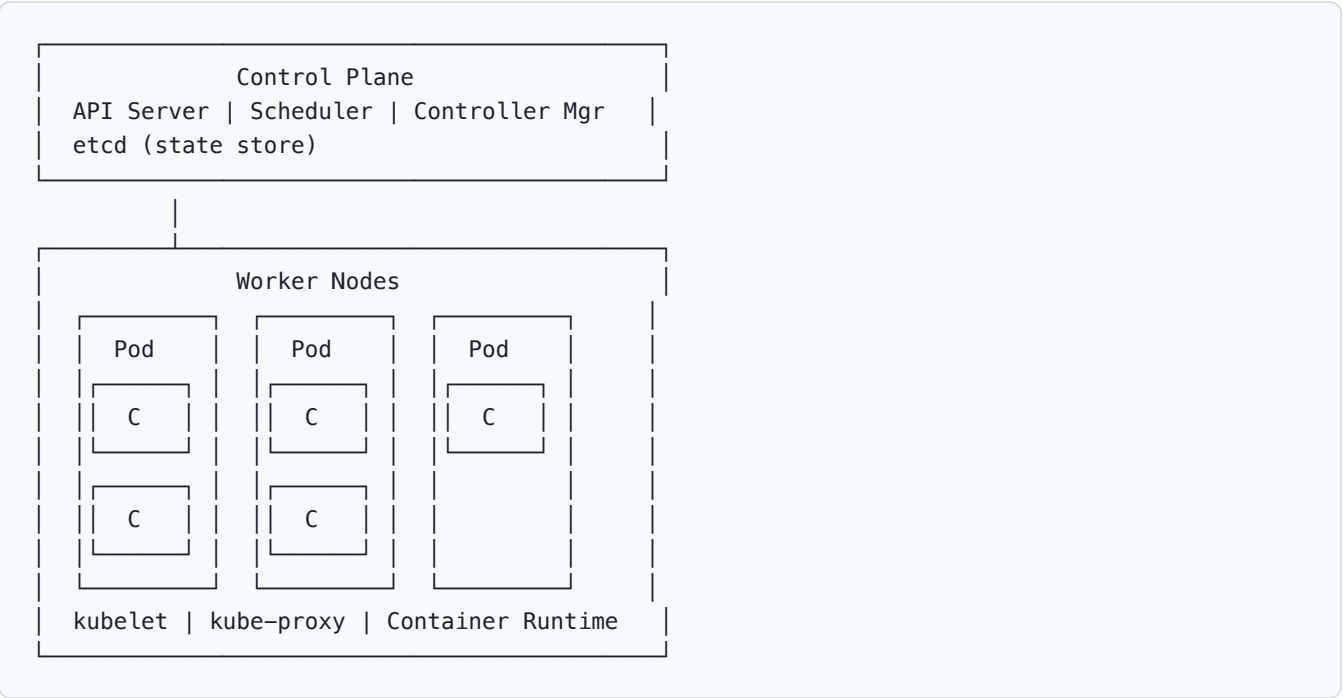
- Usar .dockerignore
- Pinned versions en base images

Docker Compose

Uso	Descripción
Local dev	Multi-service environments
Testing	Integration test environments
Demo	Quick-start setups

Kubernetes (K8s)

Arquitectura



Core Resources

Resource	Descripción	Use
Pod	Unidad mínima, 1+ containers	Deploy unit
Deployment	Replica management, rolling updates	Stateless apps
StatefulSet	Ordered deployment, stable identity	Databases, queues
Service	Internal load balancing, stable IP	Service discovery

Resource	Descripción	Use
Ingress	HTTP routing, TLS termination	External access
ConfigMap	Non-sensitive configuration	App config
Secret	Sensitive data (base64)	Credentials, keys
PersistentVolume	Storage abstraction	Stateful data
Namespace	Logical isolation	Multi-tenancy

Networking

Service Type	Exposición	Use Case
ClusterIP	Interno only	Default, inter-service
NodePort	Static port en node	Dev/testing
LoadBalancer	Cloud LB	External production
ExternalName	DNS CNAME	External service reference

Scheduling

Feature	Descripción
nodeSelector	Simple label matching
Affinity/Anti-affinity	Complex placement rules
Taints/Tolerations	Pod eviction rules
Resource requests/limits	CPU/memory guarantees
Pod Disruption Budget	Min available during disruptions

Helm

Aspecto	Detalle
Qué es	Package manager for Kubernetes
Charts	Templates + values + metadata
Release	Deployed instance of a chart
Repository	Helm chart registry

```
# values.yaml
replicaCount: 3
```

```
image:
  repository: myapp
  tag: "1.2.0"
resources:
  requests:
    cpu: 250m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
ingress:
  enabled: true
  hosts:
    - host: myapp.example.com
```

Kustomize

Aspecto	Detalle
Qué es	Patching sin templates
Base	Common YAML files
Overlays	Environment-specific patches
Built-in	kubectl apply -k

Kustomize vs Helm:

Factor	Helm	Kustomize
Templates	Go templates	JSON patches
Packaging	Charts	Directory structure
Learning curve	Media	Baja
Flexibility		
Ecosystem		
State management	Releases	In-place

Operators

Aspecto	Detalle
Qué es	Custom controllers con domain knowledge
Pattern	Reconciliation loop
Use cases	Databases, message queues, cert management

Aspecto	Detalle
Framework	Operator SDK, Kubebuilder

Ejemplos populares:

- Prometheus Operator (kube-prometheus-stack)
- PostgreSQL Operator (CloudNativePG)
- Cert-Manager
- Strimzi (Kafka)

Security

Concernión	Herramienta
Policy enforcement	OPA Gatekeeper, Kyverno
Image scanning	Trivy, Snyk, Grype
Runtime security	Falco, Tetragon
Secret management	Vault, SOPS, Sealed Secrets
Network policies	Calico, Cilium
RBAC	Kubernetes native

Managed Kubernetes

Provider	Service	Control Plane Cost
AWS	EKS	\$0.10/hr
Azure	AKS	Free (control plane)
GCP	GKE	Free standard tier
DigitalOcean	DOKS	\$12/mo
Linode	LKE	Free

Referencia: Plataformas Cloud

Local Development

Herramienta	Plataforma	Speed
minikube	Any	Medio
kind	Docker	Rápido
k3d	Docker	Rápido

Herramienta	Plataforma	Speed
Docker Desktop	Mac/Win	
Tilt	Dev tooling	Hot reload
Scaffold	Dev tooling	
Telepresence	Proxy local↔cluster	

Monitoring Kubernetes

- Instalar kube-prometheus-stack (Prometheus Operator)
- Dashboard: Grafana kubernetes dashboards
- Metrics Server para HPA
- Referencia: Monitoring stack

Decision Framework

¿Empiezas desde cero?	→ Managed K8s (EKS/GKE/AKS)
¿Local dev?	→ kind o k3d
¿Simple deployments?	→ Helm charts
¿Custom patching?	→ Kustomize
¿Stateful workloads?	→ Operators
¿Policy enforcement?	→ OPA Gatekeeper

Integración

- **IaC**: Iac terraform — Provisionar clusters
- **Cloud**: Plataformas cloud — Managed Kubernetes
- **CI/CD**: Herramientas ci cd — Deploy to K8s
- **Monitoring**: Herramientas monitoring — Observability stack
- **Testing**: Herramientas testing — Integration environments

Iac Terraform

Visión General

Infrastructure as Code (IaC) permite gestionar infraestructura de forma declarativa, versionada y reproducible. Es componente esencial de plataformas cloud y containers.

Herramientas Principales

Terraform

Aspecto	Detalle
Creator	HashiCorp
Language	HCL (HashiCorp Configuration Language)
State	State file (local o remoto)
Providers	3,000+ (AWS, Azure, GCP, K8s, etc.)
Plan	terraform plan — dry-run
Apply	terraform apply — execute
Destroy	terraform destroy — cleanup
License	BSL 1.1 (post-AUG 2023)

```
# Ejemplo Terraform
resource "aws_instance" "web" {
  ami          = "ami-0c55b159cbfaffe1f0"
  instance_type = "t3.micro"

  tags = {
    Name = "WebServer"
  }
}

resource "aws_security_group" "web_sg" {
  name = "web-sg"

  ingress {
    from_port = 80
    to_port   = 80
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

```
}  
}
```

Módulos recomendados:

- terraform-aws-modules/vpc
- terraform-aws-modules/eks
- terraform-aws-modules/rds

State Management:

- S3 + DynamoDB (AWS)
- Azure Storage
- GCS + Cloud Spanner
- Terraform Cloud

Pulumi

Aspecto	Detalle
Language	TypeScript, Python, Go, C#, Java
Model	Imperativo + declarativo
State	Pulumi Cloud o self-managed
Providers	150+ (same as Terraform)
License	Apache 2.0
Secrets	Built-in encryption

```
// Pulumi TypeScript example  
import * as aws from "@pulumi/aws";  
  
const server = new aws.ec2.Instance("web-server", {  
    ami: "ami-0c55b159cbfafa1f0",  
    instanceType: "t3.micro",  
    tags: { Name: "WebServer" },  
});
```

Ventaja sobre Terraform: Lenguajes de programación completos, testing nativo, type safety.

Ansible

Aspecto	Detalle
Modelo	Imperativo, agentless
Language	YAML (playbooks) + Jinja2

Aspecto	Detalle
Execution	SSH (push-based)
State	Stateless
Use cases	Configuration management, provisioning
AWX/Tower	UI + RBAC + orchestration
License	GPL (core), Ansible Automation Platform (paid)

```
# Ansible playbook
- hosts: webservers
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
```

CloudFormation (AWS)

Aspecto	Detalle
Provider	AWS only
Language	YAML/JSON
State	Managed by AWS
Integration	Native AWS services
Rollback	Automatic
Drift detection	Built-in

Comparison Matrix

Feature	Terraform	Pulumi	Ansible	CloudFormation
Type	Declarative	Imperative+Declarative	Imperative	Declarative
Language	HCL	Real languages	YAML	YAML/JSON
State	File-based	Cloud/managed	Stateless	AWS-managed

Feature	Terraform	Pulumi	Ansible	CloudFormation
Multi-cloud				✗ (AWS only)
Learning curve	Media	Baja (si conoces lenguaje)	Baja	Media
Testing	⚙️ (terratest)	(native)	⚙️	⚙️
Drift detection			✗	
Rollback	Manual	⚙️	Manual	Auto
Maturity				
Ecosystem				

Decision Framework

Multi-cloud + maturity?	→ Terraform
Developer-first?	→ Pulumi
Config management + prov?	→ Ansible
AWS-only + native?	→ CloudFormation
Both infra + config?	→ Terraform + Ansible
All-in-one?	→ Pulumi (with Ansible for config)

Best Practices

State Management

- Remote state (nunca local en team settings)
- State locking (DynamoDB, Pulumi Cloud)
- Encryption at rest
- Separate state files por environment
- Referencia: Cloud strategy

Code Organization

```
infra/
  modules/
    vpc/
    eks/
    rds/
  environments/
    dev/
    staging/
    prod/
  policies/
    .sentinel (Terraform) / (OPA for Pulumi)
```

Security

- Secrets in Vault, never in code
- Least privilege for IaC identity
- Policy as Code (Sentinel, OPA, Kyverno)
- Drift detection automático
- Referencia: Security practices

Testing

- **Plan verification**: Check plan output antes de apply
- **Policy testing**: Sentinel/OPA policies
- **Integration testing**: Terratest, Pulumi Testing
- **Drift detection**: Periodic reconciliation
- Referencia: Testing tools

GitOps con IaC

Git commit → CI pipeline → terraform plan → PR review → terraform apply → drift detection

- Plan output en PR comments
- Require approval antes de apply
- Auto-drift detection (diario)
- Referencia: CI/CD pipelines

Migration Strategies

De ClickOps a IaC

1. **Terraformer** — Import existing infrastructure
2. **Former2** — Generate IaC from existing resources
3. **Manual import** — terraform import / pulumi import

Entre herramientas

1. **Terraform** → **Pulumi**: Import existing state
2. **CF** → **Terraform**: CloudFormer + refactor
3. **Ansible** → **Terraform**: Separate infra from config

Referencia: Modernización legacy

Integración

- **Cloud**: Plataformas cloud — Providers cloud
- **Containers**: Container kubernetes — EKS/GKE provisioning
- **CI/CD**: Herramientas ci cd — Pipeline automation

- **Monitoring:** Herramientas monitoring — Infra metrics
- **Security:** Security practices

Plataformas Cloud

Visión General

Las plataformas cloud son el foundation para workloads modernos. La elección de provider impacta costos, managed services, IaC, y team capabilities.

Los Tres Grandes

Amazon Web Services (AWS)

Aspecto	Detalle
Market share	~31% (leader)
Services	200+ managed services
Regions	34+ regions, 108+ AZs
Free tier	12 months + always free

Servicios clave:

Categoría	Servicios
Compute	EC2, Lambda, ECS, EKS, Fargate
Storage	S3, EBS, EFS, Glacier
Database	RDS, DynamoDB, Aurora, ElastiCache
Networking	VPC, CloudFront, Route 53, ALB/NLB
AI/ML	SageMaker, Bedrock, Rekognition
Analytics	Athena, Glue, EMR, Kinesis
DevOps	CodePipeline, CodeBuild, CodeDeploy

Fortalezas: Broadest service portfolio, largest community, most mature.

Debilidades: Pricing complexity, steep learning curve, console UX.

Microsoft Azure

Aspecto	Detalle
Market share	~24%
Services	200+ services

Aspecto	Detalle
Regions	60+ regions
Strength	Enterprise integration (.NET, Active Directory, Office 365)

Servicios clave:

Categoría	Servicios
Compute	VMs, AKS, Azure Functions, Container Apps
Storage	Blob Storage, Azure Files, Managed Disks
Database	Azure SQL, Cosmos DB, PostgreSQL Flexible
Networking	VNet, Front Door, Traffic Manager
AI/ML	Azure OpenAI, Cognitive Services
Enterprise	Active Directory, DevOps, Logic Apps

Fortalezas: Hybrid cloud (Arc), .NET ecosystem, enterprise agreements, OpenAI partnership.

Debilidades: Service naming inconsistency, portal complexity.

Google Cloud Platform (GCP)

Aspecto	Detalle
Market share	~11%
Strengths	Data/ML, Kubernetes native, networking
Kubernetes	Created K8s, best managed (GKE)
AI/ML	Vertex AI, TPUs, Gemini

Servicios clave:

Categoría	Servicios
Compute	Compute Engine, GKE, Cloud Run, Functions
Storage	Cloud Storage, Persistent Disk
Database	Cloud SQL, Firestore, Spanner, BigQuery
Networking	VPC, Cloud CDN, Cloud Armor
AI/ML	Vertex AI, TPUs, BigQuery ML
Data	BigQuery, Dataflow, Pub/Sub, Dataproc

Fortalezas: Best Kubernetes (GKE), BigQuery (analytics), networking, open-source commitment.

Debilidades: Smaller enterprise footprint, fewer regions than AWS.

Comparison Matrix

Feature	AWS	Azure	GCP
Market share	(~31%)	(~24%)	 (~11%)
Service breadth			
K8s managed	EKS	AKS	GKE
Serverless	Lambda	Functions	Cloud Run
AI/ML	SageMaker, Bedrock	Azure OpenAI	Vertex AI
Data analytics	Athena, Redshift	Synapse	BigQuery
Hybrid	Outposts	Arc	Anthos
.NET support			
Pricing	Complex	Complex	Simpler
Free tier			
Open source			

Serverless Options

Service	Provider	Use Case
AWS Lambda	AWS	Event-driven, API
Azure Functions	Azure	Event-driven, .NET
Google Cloud Run	GCP	Container-based, HTTP
AWS Fargate	AWS	Container without management
Azure Container Apps	Azure	Microservices
AWS App Runner	AWS	Simplified container deploy

Serverless Decision:

Event-driven + AWS? → Lambda

Container-based? → Cloud Run

Microservices + Azure? → Container Apps

Full control containers? → Fargate

Multi-Cloud Strategy

Cuándo Multi-Cloud

Scenario	Approach
Best of breed	Select best service per use case
Risk mitigation	Avoid vendor lock-in
Regulatory	Data residency requirements
M&A	Different providers por company
Performance	Edge deployment

Herramientas Multi-Cloud

Herramienta	Tipo
Terraform/Pulumi	IaC multi-cloud
Kubernetes	Orchestration portable
OpenTelemetry	Observability vendor-neutral
Terraform Cloud	Multi-cloud state management

Multi-Cloud Challenges

- Network complexity (peering, VPN)
- Identity federation across providers
- Cost management tools per provider
- Skills split del equipo
- Referencia: IaC multi-cloud

Cloud Cost Management

Herramienta	Tipo
AWS Cost Explorer	Built-in
Azure Cost Management	Built-in
GCP Cost Tools	Built-in
Infracost	IaC cost estimation
Kubecost	Kubernetes cost allocation
CloudHealth	Multi-cloud

Herramienta	Tipo
Spot.io	Spot instance management

Referencia: Cost metrics

Cloud Migration Strategies (6 Rs)

Strategy	Descripción	Ejemplo
Rehost	Lift and shift	VM → EC2
Replatform	Lift and reshape	VM → RDS
Repurchase	SaaS replacement	On-prem → Salesforce
Refactor	Re-architect	Monolith → Lambda
Retain	Keep on-prem	Legacy systems
Retire	Decommission	Unused systems

Referencia: Modernización legacy

Decision Framework

Enterprise + Microsoft ecosystem? → Azure
Data/ML + Kubernetes focus? → GCP
Broadest services + largest community? → AWS
Multi-cloud? → Primary + secondary
Budget constrained? → Compare spot/preemptible pricing

Security y Compliance

- Shared responsibility model
- Compliance certifications: SOC2, HIPAA, PCI-DSS, GDPR
- Cloud security posture management (CSPM)
- Identity: AWS IAM, Azure AD, GCP IAM
- Referencia: Security practices

Integración

- **IaC**: Iac terraform — Provisionar infraestructura
- **Containers**: Container kubernetes — Managed K8s
- **CI/CD**: Herramientas ci cd — Deploy pipelines
- **Monitoring**: Herramientas monitoring — Cloud observability
- **AI**: AI services

Ai Coding Assistants

Visión General

AI coding assistants han transformado la productividad del developer. Estas herramientas usan LLMs para autocomplete, chat, code generation, y agentic workflows. Son el pilar principal de la IA en software engineering.

Panorama de Herramientas

GitHub Copilot

Aspecto	Detalle
Provider	GitHub (Microsoft)
Modelo	GPT-4o, Claude, Gemini (multi-model)
Pricing	Individual (\$10/mes), Business (\$19/user/mes), Enterprise (\$39/user/mes)
Features	Autocomplete, Chat, Copilot Workspace, Code Review
IDE Support	VS Code, JetBrains, Neovim, Visual Studio
Codebase context	Repo indexing (Enterprise)
Agentic	Copilot Workspace (multi-file changes)

Features clave:

- **Autocomplete:** Suggestions en tiempo real
- **Chat:** Q&A sobre código, debugging, explanations
- **Extensions:** Terminal, PR summaries, code review
- **Copilot Workspace:** Planning → implementation multi-step
- **Custom instructions:** Project-specific context

Cursor

Aspecto	Detalle
Type	AI-native IDE (fork de VS Code)
Pricing	Free (2000 completions), Pro (\$20/mes), Business (\$40/user/mes)
Models	GPT-4o, Claude 3.5/Opus, custom
Features	Tab autocomplete, Chat, Composer, codebase indexing
Differentiator	Full codebase awareness, multi-file editing

Aspecto	Detalle
Context	Automatic (entire project indexed)

Features clave:

- **Composer:** Agentic multi-file changes con planificación
- **Tab:** Context-aware autocomplete que entiende patrones
- **Codebase indexing:** Pregunta sobre cualquier archivo del proyecto
- **Privacy mode:** No training on your code

Sourcegraph Cody

Aspecto	Detalle
Provider	Sourcegraph
Pricing	Free, Pro (\$9/mes), Enterprise
Context	Code graph (entire codebase)
Strength	Enterprise code search + AI
Models	Claude, GPT-4, Gemini

Features clave:

- **Code search:** Enterprise codebase exploration
- **Context engine:** Smart context retrieval
- **Agentic:** Multi-repo changes
- **Sourcegraph integration:** Code intelligence

Codeium / Windsurf

Aspecto	Detalle
Pricing	Free (generous), Pro (\$10/mes)
Features	Autocomplete, Chat, Extensions
IDE Support	VS Code, JetBrains, Vim, +15 more
Unique	Windsurf IDE (agentic, Cascade flows)
Models	Proprietary + third-party

Features clave:

- **Free tier:** Más generoso que competidores
- **Wide IDE support:** +15 editores
- **Cascade:** Multi-step agentic reasoning

Tabnine

Aspecto	Detalle
Type	Autocomplete-focused assistant
Pricing	Free, Dev (\$12/mes), Enterprise
Focus	Code completion quality
Models	Proprietary (trained per language)
Self-hosted	Enterprise
Privacy	Code never stored

Features clave:

- **Code completion:** High-quality, low-latency
- **Self-hosted:** Enterprise on-premise option
- **Team training:** Adapta al código del equipo

Amazon CodeWhisperer / Q Developer

Aspecto	Detalle
Provider	AWS
Pricing	Free tier, Pro (\$19/user/mes)
Focus	AWS integration, security scanning
Features	Autocomplete, Chat, security scans
AWS services	Deep integration

Comparison Matrix

Feature	Copilot	Cursor	Cody	Codeium	Tabnine	Q Developer
Autocomplete						
Chat					⚙️	
Agentic (multi-file)					❌	⚙️
Codebase context	(Enterprise)				⚙️	
Free tier						
Self-hosted	(Enterprise)	❌		❌		(Q)
Multi-IDE		❌ (VS Code fork)				

Feature	Copilot	Cursor	Cody	Codeium	Tabnine	Q Developer
Security scanning		✗		✗	✗	
Privacy mode						

Impacto en Productividad

Datos de Productivity

Métrica	Impacto reportado
Code completion acceptance	30-40% suggestions accepted
Time to first working code	25-55% faster
Bug detection	+15-20% found earlier
Code review efficiency	20-30% faster reviews

Best Practices de Uso

1. **Provide context:** Descriptive variable names, comments, docstrings
2. **Review suggestions:** Nunca aceptar sin entender
3. **Use chat:** Preguntar "why" no solo "what"
4. **Custom instructions:** Configurar project context
5. **Test everything:** AI genera código plausible pero no siempre correcto
6. **Team alignment:** Establecer conventions de AI usage

Selection Framework

¿Budget + enterprise features?	→ GitHub Copilot Business
¿Maximum AI power + codebase?	→ Cursor
¿Enterprise code search + AI?	→ Cody + Sourcegraph
¿Budget constrained?	→ Codeium (free)
¿Privacy-first + self-hosted?	→ Tabnine Enterprise
¿AWS ecosystem?	→ Q Developer

Privacy y Security Considerations

Concern	Mitigation
Code sent to cloud	Self-hosted options (Tabnine)
Training on your code	Opt-out options, privacy modes
Secret leakage	Pre-commit scanning, .gitignore patterns

Concern	Mitigation
License compliance	Review generated code licenses
Data retention	Enterprise plans with zero-retention

Enterprise Adoption

Planning

1. **Pilot program**: 2-4 weeks con selected team
2. **Metrics baseline**: Measure before/after
3. **Security review**: Evaluate data flow
4. **Policy creation**: Acceptable use guidelines
5. **Training**: Team workshops on effective prompting

Rollout

1. Start with high-trust teams
2. Establish feedback loops
3. Measure productivity metrics
4. Iterate on policies
5. Expand gradually

Referencia: Métricas de productividad

Tendencias

- **Multi-model**: Elegir modelo por tarea
- **Agentic workflows**: AI que ejecuta tareas completas
- **Codebase-aware**: Contexto del proyecto completo
- **Specialized assistants**: AI por domain (testing, security, docs)
- **On-device models**: Privacy, latency improvement

Referencia: IA en Software Engineering

Integración

- **Code Review**: Ai code review — AI-assisted PR reviews
- **Testing**: Ai testing tools — AI-generated tests
- **IDEs**: Ides editores — Environment setup
- **CI/CD**: Herramientas ci cd — AI in pipelines
- **DX**: Developer experience — AI as DX component

Ai Testing Tools

Visión General

Las herramientas de testing con IA complementan las herramientas tradicionales con generación automática de tests, visual testing inteligente, y self-healing tests. Son parte de la estrategia de IA en software.

Categorías

AI-Powered Test Generation

Copilot Testing (GitHub Copilot)

Aspecto	Detalle
Integration	VS Code, JetBrains
Capabilities	Generate test cases from code, suggest assertions
Languages	Multi-lenguaje
Approach	Inline suggestions + chat
Cost	Included in Copilot subscription

Uso:

```
# Type a test function name, Copilot generates the body
def test_user_registration_with_valid_email():
    # Copilot suggests:
    user = register_user("test@example.com", "password123")
    assert user.email == "test@example.com"
    assert user.is_active is True
```

Diffblue Cover

Aspecto	Detalle
Language	Java (JVM)
Type	Unit test generation
Approach	AI-driven automatic test creation
Output	JUnit tests with assertions
Integration	IntelliJ plugin, CLI, CI/CD
Cost	Enterprise pricing

Capabilities:

- Genera unit tests automáticamente
- Coverage increase without manual effort
- No requiere setup ni configuration
- Regression test generation
- Refactoring-safe tests

Visual Testing con AI

Applitools

Aspecto	Detalle
Type	Visual AI testing
Technology	Visual AI (not pixel comparison)
Platforms	Web, Mobile, Desktop
Integrations	Selenium, Cypress, Playwright, Appium
Features	Ultrafast Grid, cross-browser
Cost	Free tier + paid plans

Features clave:

- **Visual AI:** Detecta cambios relevantes, ignora irrelevantes
- **Cross-browser:** Test visual en múltiples browsers
- **Cross-device:** Responsive testing
- **Root cause analysis:** Identifies which code change caused visual diff

Comparison visual testing:

Feature	Applitools	Percy	Chromatic	Playwright built-in
AI visual comparison				✗ (pixel-based)
Cross-browser			✗	
Smart ignores				✗
Speed	Ultrafast			
Pricing	Paid	Paid	Paid	Free

AI-Powered Test Management

Testim

Aspecto	Detalle
Type	AI-powered E2E testing
Approach	Smart locators, self-healing
Recording	Visual test recorder
Stability	AI stabilization of flaky tests
Integration	CI/CD, Selenium Grid
Cost	Enterprise pricing

Features clave:

- **Smart locators:** AI finds elements even when DOM changes
- **Self-healing:** Tests auto-adapt to UI changes
- **Visual editor:** Non-technical test creation
- **Stability score:** Measures test reliability

Mabl

Aspecto	Detalle
Type	Intelligent test automation
Approach	Low-code + AI
Features	Auto-healing, visual testing, performance
Deployment	SaaS (cloud-based runners)
Integration	CI/CD, Jira, Slack
Pricing	Per test execution

Features clave:

- **Auto-healing:** Tests adapt to application changes
- **Journey testing:** End-to-end user flows
- **Performance assertions:** Response time checks
- **Environment management:** Dynamic test environments

AI Code Analysis para Testing

CodiumAI / Qodo

Aspecto	Detalle
Type	AI test generation + code behavior analysis

Aspecto	Detalle
Approach	Analyze code behavior, generate tests
Languages	Python, JavaScript, TypeScript, Java
Features	Behavior coverage, test suggestions
Integration	VS Code, JetBrains
Cost	Free tier, Pro

Features clave:

- **Behavior analysis:** Understands code behavior, not just syntax
- **Test suggestions:** Multiple test approaches per function
- **Coverage gaps:** Identifies untested behaviors
- **Test suites:** Organized, comprehensive test generation

Comparison Matrix

Feature	Copilot	Diffblue	Applitools	Testim	Mabl	Qodo
Unit test gen			✗	✗	✗	
E2E test gen	⚙️	✗	✗			✗
Visual testing	✗	✗		⚙️		✗
Self-healing	✗	✗	✗			✗
Multi-language		✗ (Java)				⚙️
CI/CD integration	⚙️					⚙️
Learning curve	Baja	Baja	Media	Media	Baja	Baja
Cost	\$\$	\$\$\$	\$\$\$	\$\$\$	\$\$\$	\$

AI in Traditional Testing Tools

Playwright + AI

- **Playwright MCP Server:** AI can drive browser tests
- **Codegen:** Record + generate scripts
- **Trace Viewer:** AI-assisted debugging

Selenium + AI

- **Selenium IDE:** AI-enhanced recording
- **Healenium:** Self-healing for Selenium tests

Cypress + AI

- **Cypress AI plugins:** Visual AI integration
- **Component testing + AI:** Copilot integration

Selection Framework

Need unit test generation?	→ Copilot (simple) / Diffblue (Java)
Need visual testing?	→ AppliTools (best AI) / Percy
Need E2E with self-healing?	→ Testim o Mabl
Need test behavior analysis?	→ Qodo
Budget constrained?	→ Copilot + Playwright
Enterprise QA team?	→ Testim o Mabl

ROI de AI Testing

Metric	Before AI	After AI	Improvement
Test creation time	100%	40-60%	40-60% faster
Flaky test rate	15-20%	5-8%	60% reduction
Visual regression bugs	Manual detection	Auto-detected	Near 100%
Maintenance effort	High	Medium	30-50% reduction

Referencia: Testing metrics

Best Practices

1. **AI augmenta, no reemplaza:** Tests still need human judgment
2. **Review AI-generated tests:** Validate assertions are meaningful
3. **Combine approaches:** AI gen + manual exploratory
4. **Measure coverage:** Track AI vs manual test quality
5. **Iterate:** Refine AI prompts and configurations

Referencia: Testing strategies

Integración

- **Traditional testing:** Herramientas testing — Foundation testing tools
- **AI Assistants:** AI coding assistants — Code generation
- **Code Review:** AI code review — Quality gates
- **CI/CD:** Herramientas ci cd — Pipeline integration
- **AI Strategy:** AI in software

Ai Code Review

Visión General

AI code review tools automatizan parte del code review process, detectando bugs, security issues, y code smells antes de que humanos revisen. Complementan, no reemplazan, la revisión humana.

Herramientas Principales

CodeRabbit

Aspecto	Detalle
Type	AI code review bot
Integration	GitHub PR, GitLab MR
Approach	LLM-powered review with context
Features	PR review, code suggestions, security
Pricing	Free tier, Pro (\$12/mes)
Learning	Learns from team's feedback

Features clave:

- **Line-by-line review:** Comentarios específicos por línea
- **Walk-through:** Resumen ejecutivo del PR
- **Security analysis:** Detección de vulnerabilities
- **Style enforcement:** Consistencia de código
- **Fix suggestions:** Propone correcciones directas
- **Chat interactivity:** Ask follow-up questions

Qodo (formerly CodiumAI)

Aspecto	Detalle
Type	AI code analysis + review
Approach	Behavior-focused analysis
Features	Test generation, code review, PR agent
Integration	GitHub, IDE plugins
Pricing	Free, Pro

Features clave:

- **PR Agent:** Automated review en GitHub/GitLab
- **Behavior coverage:** Analiza completeness del código
- **Code suggestions:** Propuestas de mejora
- **Test suggestions:** Genera tests alongside review
- **Multiple review types:** Security, performance, style

SonarQube (with AI features)

Aspecto	Detalle
Type	Static analysis + AI
Model	Self-hosted + Cloud
Rules	5,000+ rules (multiple languages)
AI Features	Smart fixes, code explanations
Integration	CI/CD, IDE, GitHub
Pricing	Community (free), Developer (\$150/yr), Enterprise

Features clave:

- **Quality gates:** Pass/fail en CI/CD
- **Security hotspots:** Prioritized security issues
- **Code smells:** Maintainability analysis
- **Coverage tracking:** Test coverage trends
- **AI-powered fixes:** Suggests remediation

Codacy

Aspecto	Detalle
Type	Automated code review
Languages	40+ languages
Features	Static analysis, security, patterns
Integration	GitHub, GitLab, Bitbucket
Pricing	Free tier, Pro, Enterprise

DeepSource

Aspecto	Detalle
Type	Automated code review + fix

Aspecto	Detalle
Features	130+ analyzers, auto-fix PRs
Integration	GitHub, GitLab
Autofix	Auto-generated fix PRs
Pricing	Free for open-source, paid for teams

Amazon CodeGuru

Aspecto	Detalle
Type	AI code review (AWS)
Focus	Performance, security, best practices
Languages	Java, Python
Integration	GitHub, CodeCommit, Bitbucket

Comparison Matrix

Feature	CodeRabbit	Qodo	SonarQube	Codacy	DeepSource
AI review				⚙️	⚙️
Security analysis					
Auto-fix suggestions			⚙️	⚙️	
Test generation	❌		❌	❌	❌
Learning from team		⚙️	⚙️	❌	❌
Self-hosted	❌				❌
Free tier					
Multi-language		⚙️			
PR integration			⚙️		

Code Review Workflow with AI

Developer pushes code

- AI review (automatic)
- Security scan
- Style check
- Logic review
- Test suggestions

- Human review (focused on architecture, design, business logic)
- Merge

Best Practices

1. **AI first, human after:** Let AI catch obvious issues first
2. **Configure rules:** Customize AI rules for team conventions
3. **Focus human review:** Architecture, design patterns, business logic
4. **Feedback loop:** Train AI with team preferences
5. **Don't skip human review:** AI catches syntax; humans catch design

Integration with Code Review Process

GitHub Pull Request

```
# .github/workflows/review.yml
name: AI Review
on: [pull_request]
jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: coderabbit-ai/ai-pr-reviewer@latest
        with:
          review_comment: true
```

Pre-commit Integration

```
# .pre-commit-config.yaml
repos:
  - repo: https://github.com/codium-ai/pr-agent
    hooks:
      - id: pr-agent
```

Impact Metrics

Metric	Without AI	With AI	Improvement
Review turnaround	24-48h	2-4h	80%+ faster
Bugs caught in review	Baseline	+40-60%	More caught early
Security issues	Post-deploy	Pre-merge	Shift-left
Reviewer fatigue	High	Medium	Reduced noise
Time to merge	3-5 days	1-2 days	50%+ faster

Referencia: Review metrics

Security Code Review

Tool	Type	Focus
Snyk Code	SAST	Vulnerabilities
Semgrep	SAST	Pattern-based rules
CodeQL	SAST	Query-based analysis
Trivy	Container	Image vulnerabilities
OWASP ZAP	DAST	Runtime security

Referencia: Security practices

Decision Framework

Quick wins for small team?	→ CodeRabbit (easy setup)
Enterprise with test gen?	→ Qodo (PR agent + tests)
Full quality platform?	→ SonarQube + AI features
Open-source friendly?	→ DeepSource + Semgrep
Security-focused?	→ SonarQube + Snyk
All-in-one?	→ CodeRabbit + SonarQube

Adopting AI Code Review

Phase 1: Pilot (1-2 weeks)

1. Install on selected repos
2. Configure team rules
3. Measure false positive rate

Phase 2: Team rollout (2-4 weeks)

1. Train team on using AI suggestions
2. Establish feedback loop
3. Tune configuration

Phase 3: Organization (1-3 months)

1. Rollout to all repos
2. Integrate with quality gates
3. Report metrics to leadership
4. Iterate on configuration

Referencia: Evaluación framework

Integración

- **AI Assistants:** Ai coding assistants — Code generation context
- **Testing:** Ai testing tools — Generate tests during review
- **CI/CD:** Herramientas ci cd — Quality gates in pipeline
- **Practices:** Code review practices
- **Metrics:** Review quality metrics

Developer Experience

Visión General

Developer Experience (DX) mide la efectividad y satisfacción del development workflow. Herramientas DX crean Internal Developer Platforms (IDPs) que simplifican la complejidad operativa, permitiendo a developers enfocarse en product development.

DX Maturity Model

Level	Name	Characteristics
1	Ad-hoc	Cada developer configura su entorno
2	Standardized	Tools y processes documentados
3	Self-service	Developers provision resources sin tickets
4	Platform	IDP con golden paths
5	Optimized	AI-enhanced, continuous improvement

Referencia: Levels of engineering maturity

Internal Developer Platforms

Backstage (Spotify)

Aspecto	Detalle
Creator	Spotify (CNCF incubating)
Type	Developer portal framework
Tech Stack	React, TypeScript
Features	Service catalog, scaffolding, docs
Plugins	100+ community plugins
Cost	Open-source, free

Core Features:

Feature	Descripción
Software Catalog	Centralized service registry
Software Templates	Project scaffolding (golden paths)

Feature	Descripción
TechDocs	Auto-generated documentation
Search	Unified search across services
API Documentation	OpenAPI, AsyncAPI registry

Plugin Ecosystem:

- Kubernetes plugin
- CI/CD (GitHub Actions, Jenkins)
- Monitoring (Grafana, Datadog)
- Security (Snyk, Trivy)
- Cost management (Kubecost)

Port

Aspecto	Detalle
Type	Developer portal (SaaS)
Approach	No-code platform builder
Features	Self-service actions, dashboards, scorecards
Integration	GitHub, K8s, Terraform, +more
Pricing	Free tier, Team, Enterprise

Features clave:

- **Self-service actions:** Developers provision infra/resources
- **Scorecards:** Track service maturity
- **Dashboards:** Real-time visibility
- **Automation:** Workflow automation

Cortex

Aspecto	Detalle
Type	Service catalog + scorecards
Focus	Reliability and ownership
Features	Catalog, scorecards, incidents, docs

OpsLevel

Aspecto	Detalle
Type	Service ownership platform
Focus	Standards compliance
Features	Catalog, checks, ownership

Comparison Matrix

Feature	Backstage	Port	Cortex	OpsLevel
Open-source		✗	✗	✗
Self-hosted		✗ (SaaS)	✗ (SaaS)	✗ (SaaS)
Customizability			⚙	⚙
Self-service			⚙	
Scorecards	Plugin			
Learning curve	Alta	Baja	Baja	Baja
Maintenance	✗ Alto			
Plugin ecosystem			⚙	⚙

Golden Paths

Qué son

Golden paths son camino pre-aprobados y facilitados para tareas comunes:

- **Create new service:** Template con CI/CD, monitoring, docs
- **Add API endpoint:** Scaffold con validación, tests, docs
- **Deploy to production:** Self-service con approvals
- **Onboard new developer:** Automated setup

Implementación

```
# Backstage template example
apiVersion: scaffolder.backstage.io/v1beta3
kind: Template
metadata:
  name: microservice-template
  title: New Microservice
spec:
  owner: platform-team
  type: service
  steps:
```

```
- id: fetch
  action: fetch:template
  input:
    url: ./template
- id: create-github-repo
  action: github:repo:create
- id: setup-ci
  action: github:actions:workflow
- id: register-catalog
  action: catalog:register
```

Developer Portals vs CLIs vs Scripts

Approach	Pros	Cons
Portal (Backstage)	Visual, self-service, discoverable	Complex setup, maintenance
CLI tools	Fast, scriptable, composable	Learning curve, fragmented
Scripts	Quick to create, simple	No governance, hard to maintain
API platform	Programmatic, automatable	Needs client tooling

Recommendation: Start with CLI/scripts, graduate to portal as team grows.

DX Metrics

Metric	Description	Target
Time to first commit	New dev → first PR	< 1 day
Time to deploy	Code change → production	< 1 hour
Onboarding time	New hire → productive	< 1 week
Developer satisfaction	Survey score (eNPS)	> 8/10
Cognitive load	Subjective complexity	Decreasing
Environment setup time	Clone → running locally	< 10 min

Referencia: DX metrics

DORA Metrics Integration

DX tools should improve DORA metrics:

DORA Metric	DX Tool Impact
Deployment frequency	Self-service deploys
Lead time for changes	Golden paths, templates

DORA Metric	DX Tool Impact
Change failure rate	Automated quality gates
Mean time to recovery	Incident response tools

Referencia: DORA metrics

Implementation Roadmap

Phase 1: Foundation (Month 1-2)

- 1. Audit current developer pain points
- 2. Document existing processes
- 3. Set up basic service catalog (even spreadsheet)
- 4. Measure baseline DX metrics

Phase 2: Standardize (Month 3-4)

- 1. Create golden path templates
- 2. Standardize tooling choices
- 3. Document best practices
- 4. Set up CI/CD templates

Phase 3: Self-service (Month 5-8)

- 1. Deploy Backstage or Port
- 2. Implement self-service actions
- 3. Connect to infrastructure providers
- 4. Add monitoring dashboards

Phase 4: Optimize (Ongoing)

- 1. Measure DX metrics
- 2. Iterate based on feedback
- 3. Add AI-powered suggestions
- 4. Expand self-service catalog

Common DX Anti-Patterns

Anti-Pattern	Impact	Solution
Tool sprawl	Confusion, context switching	Consolidate, standardize
No golden paths	Inconsistency, wasted time	Templates, conventions
Manual provisioning	Bottlenecks, delays	Self-service platform

Anti-Pattern	Impact	Solution
No feedback loop	Stagnant DX	Metrics, surveys
Over-engineering	Complexity > value	Start simple, iterate

Integración

- **Practices:** Development practices
- **Metrics:** Measuring DX
- **CI/CD:** Herramientas ci cd — CI/CD templates
- **Containers:** Container kubernetes — Developer environments
- **Modernization:** Legacy modernizacion — DX in legacy migration
- **AI:** Ai coding assistants — AI as DX component

Legacy Modernization

Visión General

La modernización de sistemas legacy es uno de los challenges más comunes en software engineering. Estas herramientas y patrones ayudan a migrar sistemas monolíticos a arquitecturas modernas sin riesgos catastróficos.

Assessment de Legacy Systems

Herramientas de Assessment

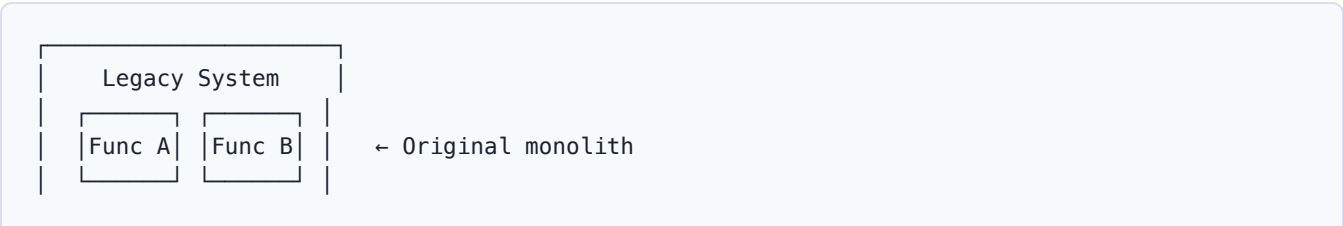
Herramienta	Tipo	Use Case
SonarQube	Static analysis	Code quality, technical debt
CAST Highlight	Portfolio analysis	Cloud readiness, risks
vFunction	AI-driven	Monolith decomposition
NServiceBus	Platform	Migration planning
OpenRewrite	Automated refactoring	Code modernization (Java)

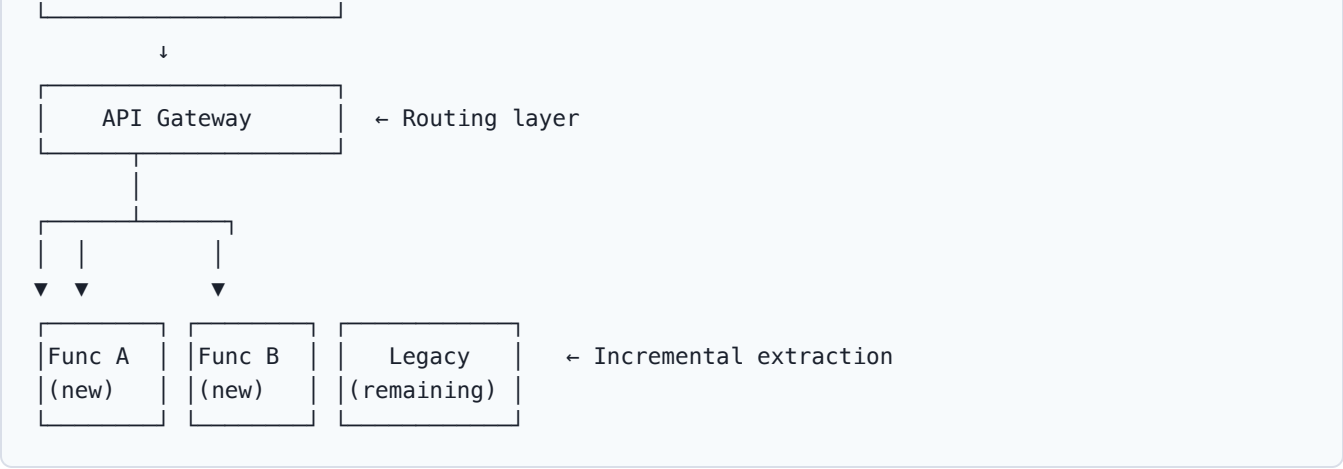
Assessment Framework

Dimension	Questions	Tools
Code quality	Debt, complexity, duplication	SonarQube, CodeClimate
Architecture	Coupling, dependencies	Structure101, Lattix
Security	Vulnerabilities, compliance	Snyk, OWASP
Performance	Bottlenecks, scaling limits	Profiling, load testing
Dependencies	Outdated libs, licenses	Snyk, Dependabot
Team knowledge	Bus factor, documentation	CODEOWNERS analysis

Migration Strategies

Strangler Fig Pattern





Pasos:

1. Identify bounded context to extract
2. Create API gateway/router
3. Build new service alongside legacy
4. Route traffic incrementally
5. Monitor and validate
6. Remove legacy component
7. Repeat for next component

The 6 Rs de Modernización

Strategy	Description	Effort	Risk
Rehost	Move to cloud as-is (lift & shift)	Bajo	Bajo
Replatform	Minor optimizations during move	Medio	Medio
Repurchase	Replace with SaaS	Bajo-Medio	Medio
Refactor/Re-architect	Cloud-native redesign	Alto	Alto
Retain	Keep as-is	N/A	N/A
Retire	Decommission	Bajo	Bajo

Referencia: Cloud migration

Automated Refactoring Tools

OpenRewrite

Aspecto	Detalle
Language	Java, Kotlin, JavaScript, Python

Aspecto	Detalle
Type	Automated refactoring
Recipes	100+ recipes
Integration	Maven, Gradle, CLI
Use cases	Migration, modernization, dependency upgrades

```
# OpenRewrite recipe example
---
type: specs.openrewrite.org/v1beta/recipe
name: com.example.MigrateToJava17
recipeList:
  - org.openrewrite.java.migrate.UpgradeToJava17
  - org.openrewrite.java.spring.boot3.UpgradeToSpringBoot3_2
```

Recipes populares:

- Java 8 → 17 migration
- Spring Boot 2 → 3 migration
- JUnit 4 → 5 migration
- javax → jakarta migration

JetBrains AI Assisted Migration

- IntelliJ refactoring + AI assistance
- Automated code transformations
- Type-safe migrations

Containerization de Legacy

Lift & Shift con Containers

Legacy Monolith → Docker Container → Managed K8s

Step	Tool	Purpose
Containerize	Docker	Package legacy app
Orchestrate	Kubernetes/EKS	Run at scale
Monitor	Prometheus/Grafana	Observability
Migrate data	AWS DMS, pg_dump	Database migration

Referencia: Containers

Database Migration

Tools

Tool	Use Case
Flyway	Schema versioning, migrations
Liquibase	Database change management
AWS DMS	Cloud database migration
pgloader	PostgreSQL migration
Alembic	Python database migrations

Migration Patterns

Pattern	Descripción
Parallel Run	Both systems run simultaneously
Truncate Switch	Copy data, switch connections
Change Data Capture	Real-time sync via CDC
Event Sourcing	Rebuild from events

Legacy Code Testing

Characterization Tests

```
// Capture legacy behavior before refactoring
@Test
void legacySystemProducesExpectedOutput() {
    LegacySystem system = new LegacySystem();
    // Capture current behavior as baseline
    Result result = system.process(input);
    assertEquals("expected output", result.toString());
}
```

Strangler Fig Testing

- **Contract tests:** Verify new service matches legacy behavior
- **Shadow traffic:** Mirror traffic to both systems
- **Canary deployment:** Gradual traffic shift
- Referencia: Testing tools

Technical Debt Management

Quantifying Debt

Metric	Description	Tool
Debt ratio	Debt / Total code	SonarQube
Remediation time	Hours to fix	SonarQube
Hot spots	Files with most changes + complexity	CodeScene
Dependency age	Outdated dependencies	Snyk, Dependabot

Prioritization Framework

High business value + Low effort → Do first
High business value + High effort → Plan carefully
Low business value + Low effort → Do when convenient
Low business value + High effort → Deprecate/avoid

Referencia: Evaluation framework

Common Patterns

Anti-Corruption Layer

```
// ACL between legacy and modern
public class OrderAdapter {
    private final LegacyOrderSystem legacySystem;

    public OrderDTO getOrder(String id) {
        LegacyOrder legacy = legacySystem.fetch(id);
        return mapToModern(legacy);
    }
}
```

Branch by Abstraction

1. Create abstraction over legacy implementation
2. Implement new version behind abstraction
3. Switch via feature flag
4. Remove old implementation

Modernization Roadmap

Phase 1: Assessment (1-2 months)

1. Code quality analysis (SonarQube)
2. Architecture review
3. Business value mapping
4. Risk assessment
5. Create modernization backlog

Phase 2: Foundation (2-4 months)

1. Set up CI/CD for legacy
2. Add characterization tests
3. Containerize monolith
4. Set up monitoring
5. Create anti-corruption layers

Phase 3: Incremental Extraction (6-18 months)

1. Identify first bounded context
2. Build new service (Strangler Fig)
3. Route traffic incrementally
4. Validate and monitor
5. Repeat for next context

Phase 4: Optimization (Ongoing)

1. Decommission legacy components
2. Optimize new architecture
3. Remove ACLs
4. Continuous improvement

Success Metrics

Metric	Target	Measurement
Legacy code coverage	Decreasing	SonarQube
Deployment frequency	Increasing	DORA metrics
Mean time to recovery	Decreasing	Monitoring
Team velocity	Increasing	Sprint metrics
System reliability	Maintained or improved	SLOs

Referencia: Migration metrics

Integración

- **IaC**: Iac terraform — Cloud provisioning

- **Cloud:** Plataformas cloud — Target platforms
- **Containers:** Container kubernetes — Containerization
- **Testing:** Herramientas testing — Validation
- **CI/CD:** Herramientas ci cd — Pipeline for legacy
- **Monitoring:** Herramientas monitoring — Observability

Evaluacion Herramientas

Visión General

Elegir herramientas correctamente es critical para la productividad del equipo y el éxito del proyecto. Este framework estandariza el proceso de evaluación, reduciendo bias y ensuring alignment con necesidades reales.

Evaluation Criteria

Technical Criteria

Criteria	Weight	Description
Functionality	25%	Does it solve the problem?
Integration	20%	Fits existing stack?
Performance	15%	Meets performance requirements?
Scalability	10%	Grows with team/project?
Security	10%	Meets security standards?
Reliability	10%	Uptime, SLA, maturity?
Extensibility	10%	Customizable, plugin ecosystem?

Business Criteria

Criteria	Weight	Description
Total Cost of Ownership	30%	License + infrastructure + maintenance + training
Vendor Stability	20%	Financial health, market position
Community/Ecosystem	20%	Docs, forums, plugins, talent pool
Learning Curve	15%	Time to productivity
Support	15%	Vendor support quality, SLAs

Team Criteria

Criteria	Weight	Description
Developer Satisfaction	25%	Survey, interviews
Skill Availability	25%	Can we hire for this?

Criteria	Weight	Description
Adoption Resistance	25%	How much change is this?
Training Resources	25%	Documentation, courses available?

Evaluation Scorecard

```
## Tool Evaluation: [Tool Name]
**Date**: [Date]
**Evaluator**: [Name]
**Use Case**: [Description]

### Scores (1-5)

| Category | Score | Weight | Weighted |
|-----|-----|-----|-----|
| Functionality | /5 | 0.25 | |
| Integration | /5 | 0.20 | |
| Performance | /5 | 0.15 | |
| Security | /5 | 0.10 | |
| Cost | /5 | 0.20 | |
| Learning Curve | /5 | 0.10 | |
| **Total** | | | **/5** |

### Pros
- [ ]
- [ ]

### Cons
- [ ]
- [ ]

### Risks
- [ ]

### Recommendation: [Adopt / Pilot / Reject]
```

Decision Matrix Template

Criteria	Weight	Tool A	Tool B	Tool C
Functionality	25%			
Integration	20%			
Performance	15%			
Security	10%			

Criteria	Weight	Tool A	Tool B	Tool C
Cost (TCO)	15%			
Community	10%			
Learning curve	5%			
Weighted Score				

PoC Process

Step 1: Define Scope (1-2 days)

Objective: [What are we evaluating?]
Success Criteria: [Measurable outcomes]
Duration: [1-2 weeks typical]
Participants: [Who is involved?]
Budget: [Time + money]

Step 2: Environment Setup (1 day)

- 1. Create isolated test environment
- 2. Set up representative data/scenarios
- 3. Ensure tool access for all participants
- 4. Document setup steps (reproducibility)

Step 3: Execution (1-2 weeks)

Day	Activity
1-2	Basic functionality testing
3-4	Integration with existing tools
5-6	Performance testing
7-8	Edge cases and limitations
9	Developer experience survey
10	Documentation review

Step 4: Evaluation (2-3 days)

- 1. Complete scorecards individually
- 2. Team discussion and calibration
- 3. Final scoring
- 4. Recommendation document

Step 5: Decision (1 day)

1. Present findings to stakeholders
2. Discuss trade-offs
3. Make decision with documented rationale
4. Plan rollout if adopted

Common Evaluation Mistakes

Mistake	Impact	Mitigation
Evaluating in isolation	Missing integration issues	Test with real stack
Over-indexing on features	Choosing complex tools	Focus on core needs
Ignoring learning curve	Low adoption	Include team in evaluation
No PoC	Wrong assumptions	Always prototype
ignoring vendor lock-in	Future flexibility loss	Evaluate exit strategy
Copying competitors	Different context	Evaluate for YOUR needs
Ignoring cost TCO	Budget surprises	Calculate full TCO

TCO Calculation

```
## Total Cost of Ownership (Annual)

### Direct Costs
- License fee: $X
- Infrastructure: $X
- Support/maintenance: $X

### Indirect Costs
- Setup time: X hours × $rate
- Training: X hours × $rate × X people
- Migration: X hours × $rate
- Productivity dip (est): X weeks × $rate

### Ongoing Costs
- Maintenance: X hours/month × $rate
- Updates/upgrades: X hours/quarter × $rate
- Admin overhead: X hours/month × $rate

### TOTAL (Year 1): $X
### TOTAL (Annual ongoing): $X
```

Technology Radar

Adopt (Use with confidence)

- Proven in production
- Team knows it well
- Strong ecosystem

Trial (Worth pursuing)

- Validated in PoC
- Not yet in production
- Promising for specific use cases

Assess (Worth researching)

- Interesting capabilities
- Needs deeper evaluation
- May fit future needs

Hold (Proceed with caution)

- Known issues or risks
- Better alternatives exist
- Wait for maturity

Referencia: Measurement framework

AI Tools Evaluation

Specific Considerations for AI Tools

Criteria	Consideration
Data privacy	Where does code go? Training policy?
Accuracy	False positive/negative rate
Model flexibility	Can you choose/change models?
Latency	Response time for real-time features
Cost model	Per-seat vs per-use vs flat
Vendor risk	AI company stability, model updates

Referencia: AI Assistants

Referencia: AI Code Review

Referencia: AI Testing Tools

Evaluation Templates by Category

IDE/Editor

- Extension ecosystem breadth
- AI integration quality
- Performance on project size
- Remote development capability
- Referencia: Ides editores

CI/CD

- Pipeline execution speed
- Self-hosted option
- Marketplace/actions ecosystem
- Security features
- Referencia: Herramientas ci cd

Testing

- Framework compatibility
- CI/CD integration
- Debugging experience
- Community and docs
- Referencia: Herramientas testing

Monitoring

- Multi-signal support (metrics, logs, traces)
- Alert flexibility
- Dashboard customization
- Cost at scale
- Referencia: Herramientas monitoring

Decision Documentation

```
# Technology Decision: [Tool Name]

## Context
- Problem being solved
- Current state
- Constraints

## Options Evaluated
| Option | Pros | Cons | Score |
|-----|-----|-----|-----|
| Tool A | | | |
| Tool B | | | |
| Tool C | | | |
```

```
## Decision
**Selected**: [Tool]
**Rationale**: [Why]
**Risks**: [Known risks]
**Mitigation**: [How to address]
**Review Date**: [When to reassess]
```

Integración

- **Metrics:** Measurement framework
- **Practices:** Decision-making practices
- **Processes:** Tool selection process
- **DX:** Developer experience — DX impact of tools
- **All tools:** Refer to specific tool files in this section

README

Visión General

Esta sección cataloga las herramientas y tecnologías clave que componen el stack moderno de software engineering. Cada herramienta se evalúa en contexto, comparada con alternativas y vinculada a las prácticas donde aporta mayor valor.

Mapa de Herramientas

Desarrollo

Herramienta	Archivo	Foco
IDEs y Editores	Ides editores	Entornos de desarrollo, AI assistants integrados
AI Coding Assistants	Ai coding assistants	Copilot, Cursor, Cody, Codeium
AI Code Review	Ai code review	CodeRabbit, Qodo, SonarQube AI
AI Testing Tools	Ai testing tools	Diffblue, Testim, Mabl, Appltools

Infraestructura

Herramienta	Archivo	Foco
Containers y Kubernetes	Container kubernetes	Docker, K8s, Helm, Operators
Infrastructure as Code	Iac terraform	Terraform, Pulumi, Ansible
Plataformas Cloud	Plataformas cloud	AWS, Azure, GCP, Serverless

Proceso y Calidad

Herramienta	Archivo	Foco
CI/CD	Herramientas ci cd	GitHub Actions, Jenkins, GitLab CI
Testing	Herramientas testing	JUnit, pytest, Jest, Cypress, Playwright
Repositorios	Gestion repositorios	GitHub, GitLab, Bitbucket
Monitoreo	Herramientas monitoring	Prometheus, Grafana, Datadog, ELK

Estrategia

Herramienta	Archivo	Foco
Developer Experience	Developer experience	Backstage, Port, DX maturity
Modernización Legacy	Legacy modernizacion	Strangler Fig, migration tools
Evaluación de Herramientas	Evaluacion herramientas	Evaluation framework, decision matrix

Cómo Usar Esta Sección

1. **Evaluando una herramienta** → Evaluacion herramientas
2. **Armando un tech stack** → Empezar por categoría relevante
3. **Comparando alternativas** → Cada archivo incluye tablas comparativas
4. **Modernizando legacy** → Legacy modernizacion
5. **Buscando mejor DX** → Developer experience

Filosofía de Selección

- Elegir herramientas que se integren entre sí
- Preferir soluciones con ecosistemas activos
- Evaluar costo total de propiedad, no solo licencia
- Considerar learning curve del equipo
- Validar con PoC antes de adoptar

Referencias

- Fundamentos de Software Engineering
- Procesos de Desarrollo
- Prácticas Recomendadas
- Métricas y KPIs
- IA en Software Engineering